🐳 Docker Networking

◈ 1. What is Docker Networking?

Docker networking enables containers to communicate:

- With each other
- With the host machine
- With the external world (internet)

◈ 2. Types of Docker Networks

Network Type	Use Case	Explanation
bridge (default)	Most common for single-host	Each container gets a private IP in a virtual network
host	Performance, uses host's IP	Container uses the host's networking directly
none	Fully isolated	No network access
overlay	Multi-host Swarm setup	Allows containers across hosts to connect

---

◈ 3. List Docker Networks

docker network ls

◈ 4. Inspect a Network

docker network inspect bridge

◈ 5. Create a Custom Bridge Network

docker network create mynet

◈ 6. Run Containers in Custom Network

Example: Web App and Database in Same Network

docker run -d --name mysql-db --network mynet mysql

docker run -d --name web-app --network mynet my-app-image

Now, web-app container can access MySQL using the hostname mysql-db.

◈ 7. Accessing Container via Port Mapping

docker run -d -p 8080:80 --name mynginx nginx

- Maps host port 8080 to container's port 80
- Visit http://localhost:8080 in browser

◈ 8. Connect an Existing Container to a Network

docker network connect mynet existing-container

◈ 9. Remove a Network

docker network rm mynet

⚠ Cannot remove a network if containers are still attached.

◈ 10. Troubleshooting Tips

- Use docker exec -it container bash to get inside a container
- Use ping container-name to check connectivity within the same network
- Always assign containers to **custom networks** for better DNS and communication

---

🧪 Docker Networking Practice Problems with Solutions

---

1 Simple Port Mapping

◈ **Task:** Run an nginx container that is accessible on your host machine at port 8081.

✅ **Solution:**

docker run -d -p 8081:80 --name webserver nginx

Visit: http://localhost:8081

---

2 Custom Network Communication

◈ **Task:**

- Create a custom bridge network my-bridge-net
- Run two containers (app1, app2) in it
- Ping from one to the other using name

✅ **Solution:**

```
docker network create my-bridge-net

docker run -dit --name app1 --network my-bridge-net alpine sh
docker run -dit --name app2 --network my-bridge-net alpine sh

docker exec -it app1 ping app2
```

---

3 Host Network Usage (Linux only)

🔹 **Task:** Run a container using host network and verify no port mapping is needed.

✅ **Solution:**

```
docker run -d --network host --name host-nginx nginx
```

Visit: http://localhost (not port-mapped, uses host network)

---

4 Database + App in Same Network

🔹 **Task:**

- Create network cloud-net
- Run MySQL and Web App in same network
- Connect app to DB via hostname

✅ **Solution:**

```
docker network create cloud-net

docker run -d --name mysql-db --network cloud-net -e MYSQL_ROOT_PASSWORD=root mysql

docker run -dit --name myapp --network cloud-net alpine sh
```

(Inside myapp container:)

```
apk add mysql-client
mysql -h mysql-db -u root -p
```

---

5 Check Container IP

🔹 **Task:** Get the IP address of a container.

✅ **Solution:**

```
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' container_name
```

Example:

```
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' webserver
```

---

6 Multiple Networks for One Container

🔷 **Task:**

- Create two networks
- Run a container in one, then connect to another

✅ **Solution:**

```
docker network create net-a
docker network create net-b

docker run -dit --name testbox --network net-a alpine sh

docker network connect net-b testbox

docker inspect testbox
```

Check networks listed under "Networks".

---

7 Isolated Container (No Network)

🔷 **Task:** Run container without network and try to ping outside.

✅ **Solution:**

```
docker run -dit --name isolated --network none alpine sh
docker exec -it isolated ping google.com
```

❌ Expected: No internet access / ping fails.

---

8 Remove a Network

🔷 **Task:**

- Create network
- Attach a container
- Stop & remove container

- Then remove network

✅ **Solution:**

```
docker network create temp-net

docker run -dit --name tempbox --network temp-net alpine sh

docker stop tempbox
docker rm tempbox

docker network rm temp-net
```

---

9 Inspect Network Configuration

🔹 **Task:** Inspect a user-defined network.

✅ **Solution:**

```
docker network inspect my-bridge-net
```

Check:

- Subnet
- Gateway
- Containers attached
- Internal IPs

🔟 **Test DNS Resolution Between Containers**

🔹 **Task:**

- Run two containers in same custom network
- Ping using container names

✅ **Solution:**

```
docker network create test-net

docker run -dit --name alpha --network test-net alpine sh
docker run -dit --name beta --network test-net alpine sh

docker exec -it alpha ping beta
```

🌐 **1. Overlay Network (Swarm Mode)**

📌 **What is it?**

An **overlay network** enables **multi-host container communication**. It lets containers running on different Docker **Swarm nodes** (hosts) communicate **as if they're on the same local network**.

✅ **Key Features:**

- Used only in **Docker Swarm mode**
- Supports **service discovery** via internal DNS
- Enables communication between containers on different physical/virtual machines
- Secure (uses encrypted tunnels)
- Used with services, not standalone containers

⚙️ **Setup Steps:**

1. Initialize Swarm

On your first node (manager):

```
docker swarm init
```

This will give you a command to join worker nodes.

2. Create an Overlay Network

```
docker network create --driver overlay --attachable my-overlay-net
```

- --attachable allows you to connect standalone containers (optional).
- Without --attachable, only **Swarm services** can use it.

3. Deploy a Service on the Overlay Network

```
docker service create --name nginx-service --network my-overlay-net -p 8080:80 nginx
```

4. Check Network

```
docker network inspect my-overlay-net
```

📘 **Use Case:**

Overlay networks are perfect for:

- Distributed microservices
- High-availability services
- Load-balanced containers across multiple nodes

▫️ 2. Macvlan Network

📌 What is it?

Macvlan gives each container its own **MAC address** and lets it appear as a **physical device** on your **local LAN**. It's ideal when:

- Containers need to be accessed directly from your LAN
- You're integrating with **legacy systems**
- Your container must be treated like a **real machine** on the network

---

✅ Key Features:

- Bypasses Docker's NAT
- Assigns **real IP addresses** from your network (e.g., 192.168.x.x)
- Makes containers visible to external devices
- **Does not support container-to-container communication** on same host (without extra config)

⚙️ Setup Steps:

1. Create Macvlan Network

```
docker network create -d macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  -o parent=eth0 \
  macvlan_net
```

- Replace eth0 with your **host's real network interface**.
- Subnet should match your LAN.
- IPs assigned to containers must be **free addresses** on your LAN.

2. Run Container with Macvlan IP

```
docker run -dit --name macvlan-nginx --network macvlan_net nginx
```

Now the container might get 192.168.1.100 and can be accessed directly from other LAN devices.

⚠️ Caveats:

- **No communication** between container and host (unless special bridge workaround is set).
- Needs careful IP and MAC management.
- Requires **root** privileges on the Docker host.

🔁 **Summary Comparison**

Feature	**Overlay (Swarm Mode)**	**Macvlan**
Scope	Multi-host (via Swarm)	Single host, direct LAN exposure
Use case	Microservices, distributed	Legacy systems, LAN access
Container IP visible	No (internal only)	Yes (visible on LAN)
Host ↔ Container	Yes	No (needs workaround)
Works without Swarm	No	Yes